Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Continuando...

Estruturas de Dados Básicas II

Recorrências


EI PSIU! ESTA INDO DORMIR???
UHUM... JÁ SÃO 03:20 DA MADRUGADA
#PONTOBROTHERS
EU ACHO QUE SEI COMO FAZER AQUELE CÓDIGO RODAR!

Anteriormente...

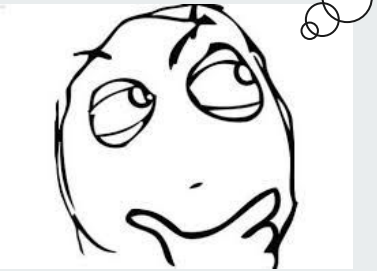
1. Implemente os códigos idadeRep e idadeRep2;
2. Crie instâncias aleatórias com idades (inteiros) variando entre 0 e 100, com tamanhos `n=100`, `n=1000`, `n=1000000`
3. Compute os tempos de processamento e compare



Definição

Uma função é recursiva se chama a si mesma.

Será que reduzir o problema em instâncias menores pode ser uma boa estratégia para resolver o problema principal?



Se o algoritmo **A** resolve uma instância **I** de tamanho **n**, será que A também resolve **I′< I**?

E será que a resolução de **I′** ajuda a resolver **I**?



PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

I
n = 10, x = ?

56	12	34	8	10	4	101	37	9	41

I'
n = 5, x = ?

56	12	34	8	10

➔ `buscaLista` **resolve** `I` **e** `I'`**?**

➔ **Isso pode ajudar na solução?**

PSEUDOCÓDIGO

```
buscaLista2(A,n,x)
i = 1
enquanto i<=n e A[i]< x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

I
n = 10, x = 37

4	8	9	10	12	34	37	41	56	101

I'
n = 5, x = 41

4	8	9	10	12

I''
n = 5, x = 37

34	37	41	56	101

➔ `buscaLista2` **resolve** `I, I' e I''`**?**

➔ **Isso pode ajudar na solução?**

Problema: Uma sequência de **n** números **A={a1, ..., an}** e uma chave de busca **x**.

- ➔ Sabemos resolver se $x \in A[n]$?
- ➔ Sabemos resolver se $x \in A[1,...,(n-1)]$?

PSEUDOCÓDIGO

```
bLRecursiva(A,n,x)
se n == 0 então
   retorne -1
se A[n]==x então
   retorne n
retorne bLRecursiva(A,(n-1),x)
```

PSEUDOCÓDIGO

```
bLRecursiva(A,n,x)
1: se n == 0 então
2:     retorne -1
3: se A[n]==x então
4:     retorne n
5: retorne bLRecursiva(A,(n-1),x)
```

Queremos Provar que **blRecursiva** retorna (-1) caso a chave de busca não esteja na lista e devolve o índice da posição do elemento, caso este esteja na lista.

Prova por Indução:

- ➔ **Caso Base**
- ➔ **Algum caso k, 0≤k<n → vale para n**

Prova por indução

Caso Base (n=0)

Neste caso, `A[0]` é vazio e é retornado (`-1`), pois o elemento não está na lista . Logo o caso base vale.

Considere algum `n > 0`
(Hipótese P(k) vale `0≤k<n`)

- Quando o algoritmo recebe um `n > 0` ele testa (linha 3) se o valor é o procurado (`A[n]==x`). Caso o teste seja bem sucedido, ele retorna o índice da posição do elemento.
- Caso o teste da linha 3 seja FALSO, é retornada a chamada recursiva da linha 5, `bLRecursiva(A,(n-1),x)`. Como, por hipótese essa chamada verifica adequadamente se `x ∈ A[1,...n-1]` e como sabemos que, `A[n] não é igual a x` então concluímos que o algoritmo opera dentro do esperado.

E como fica a análise de complexidade?

PSEUDOCÓDIGO

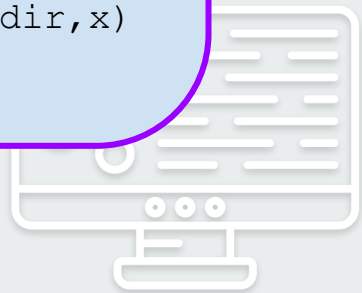
```
bLRecursiva(A,n,x)
se n == 0 então
   retorne -1
se A[n]==x então
   retorne n
retorne bLRecursiva(A,(n-1),x)
```

PSEUDOCÓDIGO

```
buscaBinaria(A,n,x)
esq = 1
dir = n
enquanto esq<dir faça
    m = ⌊(esq+dir)/2⌋
    se k > A[m] então
        esq = m+1
    senão
        dir = m
se A[esq]==x então
    retorne esq
retorne -1
```

PSEUDOCÓDIGO

```
bBinRec(A,esq,dir,x)
se esq>dir então
    retorne -1
m = ⌊(esq+dir)/2⌋
se A[m]==x então
    retorne m
se A[m] > x então
    retorne bBinRec(A,esq,m-1,x)
senão
    retorne bBinRec(A,m+1,dir,x)
```



E como fica a análise de complexidade?

PSEUDOCÓDIGO

```
bBinRec(A,esq,dir,x)
se esq>dir então
    retorne -1
m = ⌊(esq+dir)/2⌋
se A[m]==x então
    retorne m
se A[m] > x então
    retorne bBinRec(A,esq,m-1,x)
senão
    retorne bBinRec(A,m+1,dir,x)
```

Definição

Recorrência é uma expressão que descreve uma função em termos dela mesma.

Sequência de Fibonacci

- F(0) = 0
- F(1) = 1
- F(2) = F(1) + F(0) = 0+1 = 2
- F(3) = F(2) + F(1) = 2+1 = 3
- F(4) = F(3) + F(2) = 3+2 = 5
- F(5) = F(4) + F(3) = 5+3 = 8
- F(6) = F(5) + F(4) = 8+5 = 13

(...)

- F(n) = F(n-1) + F(n-2)

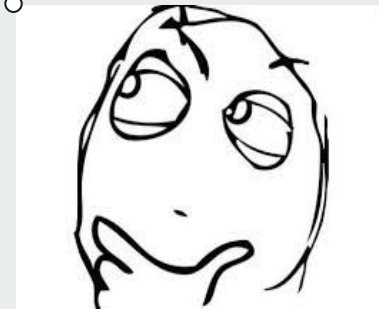
PSEUDOCÓDIGO

```
fib(n)
se n<2 então
   retorne n
senão
   retorne fib(n-1)+fib(n-2)
```

PSEUDOCÓDIGO

```
bLRecursiva(A,n,x)
se n == 0 então
   retorne -1
se A[n]==x então
   retorne n
retorne bLRecursiva(A,(n-1),x)
```

Que tal usar recorrências para descrever o tempo de execução de um algoritmo recursivo?

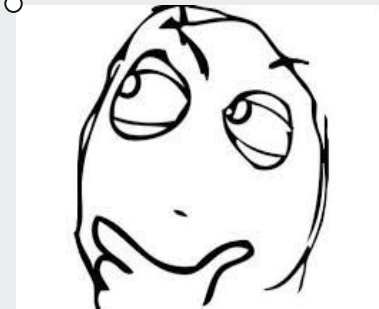


PSEUDOCÓDIGO

```
bLRecursiva(A,n,x)
se n == 0 então
   retorne -1
se A[n]==x então
   retorne n
retorne bLRecursiva(A,(n-1),x)
```

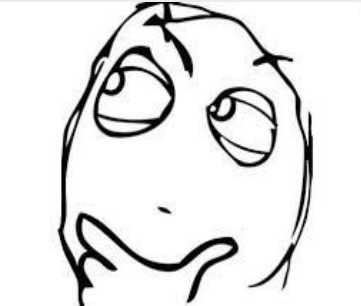
$$T(n) \leq \begin{cases} \Theta(1) \text{ se } n=0 \\ \Theta(1)+T(n-1) \end{cases}$$

Que tal usar recorrências para descrever o tempo de execução de um algoritmo recursivo?



PSEUDOCÓDIGO

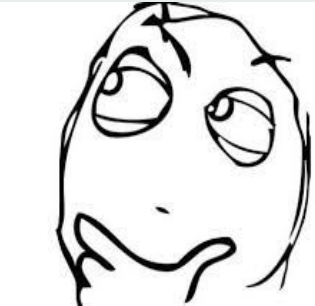
```
bBinRec(A,esq,dir,x)
se esq>dir então
    retorne -1
m = ⌊(esq+dir)/2⌋
se A[m]==x então
    retorne m
se A[m] > x então
    retorne bBinRec(A,esq,m-1,x)
senão
    retorne bBinRec(A,m+1,dir,x)
```



PSEUDOCÓDIGO

```
bBinRec(A,esq,dir,x)
se esq>dir então
    retorne -1
m = ⌊(esq+dir)/2⌋
se A[m]==x então
    retorne m
se A[m] > x então
    retorne bBinRec(A,esq,m-1,x)
senão
    retorne bBinRec(A,m+1,dir,x)
```

$$T(n) \leq \begin{cases} \theta(1) \text{ se } n=0 \\ \theta(1)+T(n/2) \end{cases}$$



$$T(n) \leq \begin{cases} \Theta(1) \text{ se } n=0 \\ \Theta(1)+T(n-1) \end{cases}$$

$$T(n) \leq \begin{cases} \Theta(1) \text{ se } n=0 \\ \Theta(1)+T(n/2) \end{cases}$$

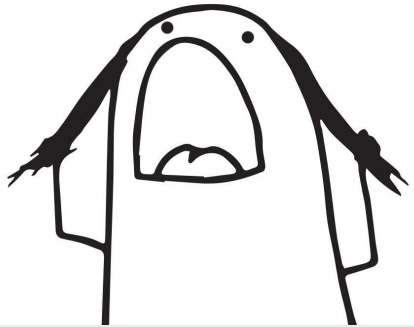
Não nos dão informações diretas sobre a complexidade em notação assintótica...

O que acharam da aula de hoje?



Para resolver as recorrências, usaremos alguns métodos

- ➔ Substituição
- ➔ Iteração
- ➔ Árvore de Recursão
- ➔ Teorema mestre



Dúvidas?!

